[El títol del TFG]

[El subtítol del tfg: Opcional]

TREBALL DE FI DE GRAU DE

Aniol Petit Cabarrocas

Jordi Taixés i Ventosa

Mathematical Engineering in Data Science

Curs 2025 - 2026

Universitat Pompeu Fabra Barcelona
Escola d'Enginyeria

Do or do not, there is no try - Master Yoda

Agraïments

dsds

Resum

[Resum aquí]

Abstract

[Abstract here]

Resumen

[Resumen aquí]

Preface

Contents

List of Figures

List of Tables

1 Introduction

2 Theoretical Preliminaries

2.1 Graph Theory Foundations

To computationally solve a routing problem across the European transit network, the physical infrastructure must be translated into a mathematical structure known as a graph. A graph $G = (V, E)$ is defined by a set of vertices V (or nodes) and a set of edges E that connect pairs of vertices. In the context of transportation routing, a standard graph is inherently directed and weighted.

Directed edges. Transit connections are unilateral. An edge $e = (u, v)$ indicates a valid route from node u to node v. The existence of $e = (u, v)$ does not imply the existence of a return edge $e' = (v, u)$ at the same time or cost.
Weighted edges. Each edge carries a cost, typically represented by a weight function $w(u, v)$. In our context, this weight represents temporal costs, such as the travel duration of a flight or the waiting time of a layover.

A path in this graph is a sequence of vertices where each adjacent pair is connected by a valid edge. The goal of a routing algorithm is to find a specific path that maximizes or minimizes a certain objective function (e.g., minimizing total weight or maximizing the number of unique vertices visited).

2.1.1 Time-Dependent vs. Time-Expanded Networks

A traditional static graph is insufficient for modeling public transit because connections do not exist continuously; they are strictly bound to specific departure and arrival schedules. To model this dimension of time, routing problems typically rely on one of two paradigms:

Time-Dependent Networks. The physical topology of the graph remains static (e.g., one vertex per station), but the weight of an edge $w(u, v, t)$ is a function of the departure time t. This keeps the network compact, but complicates the routing algorithms, as the cost of traversal fluctuates dynamically.
Time-Expanded Networks. Time is explicitly discretized and baked directly into the topology of the graph. A single physical station is duplicated into multiple vertices, each representing that station at a specific point in time (e.g., Station A at 10:00, Station A at 10:15). This dramatically increases the number of nodes and edges, but allows the use of standard, static routing algorithms, since every edge now possesses a fixed, immutable weight.

Because our problem operates within a strict 24-hour window using discrete, fixed transit schedules, modeling time efficiently is the central architectural challenge. To systematically evaluate the optimal data structure for this Guinness World Record routing problem, this thesis intentionally implements and

compares both paradigms: a memory-efficient time-dependent multi-graph (detailed in Chapter 4) and a structurally unrolled time-expanded network (detailed in Chapter 5).

2.1.2 Activity-on-Edge (AoE) vs. Activity-on-Node (AoN)

Within time-expanded models, the data can be structured in two distinct ways:

Activity-on-Edge (AoE). The traditional approach where vertices represent physical locations at specific times, and edges represent the actual transport activities (a train moving between locations) or waiting periods at a station.
Activity-on-Node (AoN). An inverted approach where the vertices themselves represent the specific transport activities (a flight with fixed departure and arrival times), and the edges represent the valid layovers or transfer opportunities connecting them.

Understanding the distinction between AoE and AoN is critical, as the choice of paradigm radically alters the branching factor, memory footprint, and traversability of the search space. A core objective of this research is to contrast these two paradigms. Chapter 4 evaluates an AoE time-dependent structure, while Chapter 5 introduces a structural pivot to an AoN framework, allowing for a direct comparative analysis of their computational viability on a continental scale.

2.2 Search and Optimization Algorithms

When graph networks become too large for brute-force enumeration, exact and heuristic optimization algorithms are required to traverse the search space efficiently.

2.2.1 Dynamic Programming and Label Setting

Dynamic Programming (DP) is a mathematical optimization method that solves complex problems by breaking them down into simpler, overlapping subproblems. In the context of graph routing, DP is often implemented via Label Setting algorithms. As the algorithm traverses the graph, it assigns "labels" to each node, where a label represents the accumulated cost or state of a specific path reaching that node.
In multi-objective optimization problems—where multiple conflicting criteria must be optimized simultaneously (e.g., minimizing time while maximizing profit)—a single optimal path cannot always be determined dynamically. Instead, the algorithm must maintain a Pareto Frontier at each node. The frontier is a set containing all non-dominated labels. A label L_1 mathematically "dominates" L_2 if it is equal or superior in all evaluated objectives, and strictly superior in at least one. Dominated labels are permanently discarded, while the non-dominated set is kept to spawn future branches.

2.2.2 Depth-First Search (DFS) and Branch-and-Bound

Depth-First Search (DFS) is a fundamental graph traversal algorithm that explores as far down a single branch of a tree as possible before backtracking. Because DFS only requires storing the nodes along the current active path, its memory complexity grows linearly with the depth of the search tree rather than exponentially with its breadth, making it highly memory-efficient.
To optimize DFS for finding absolute maximums or minimums, it is often paired with Branch-and-Bound (BnB), a paradigm for discrete and combinatorial optimization.

Branching. The algorithm systematically divides the search space by exploring subsequent child nodes.
Bounding. At each node, a bounding function calculates an optimistic mathematical estimate (an upper or lower bound) of the best possible solution that could be found by continuing down that branch.

If the calculated bound of a branch is worse than the best solution already found elsewhere in the search (the incumbent solution), the algorithm instantly abandons the branch. This technique, known as pruning, eliminates vast sections of the search space without requiring explicit evaluation.

2.3 Distributed Computing

When a computational problem's scale exceeds the memory or processing capacity of a single machine, distributed computing methodologies are employed to divide the task across a High-Performance Computing (HPC) cluster. An HPC cluster is a network of independent computers (nodes) that function as a single system. Access to these resources is governed by a workload manager or job scheduler, such as SLURM (Simple Linux Utility for Resource Management). The scheduler allocates specific hardware constraints, such as CPU cores and maximum RAM, and queues tasks to ensure that execution does not exceed physical memory limits, which would otherwise trigger immediate task termination.

To efficiently search massive combinatorial state spaces, algorithms frequently leverage data-level parallelism. In this paradigm, the exact same algorithm is executed concurrently across non-overlapping subsets of the initial data. In HPC environments, this is seamlessly orchestrated using job arrays. A job array takes a single execution script and duplicates it into dozens or hundreds of independent tasks, assigning each a unique environmental index. The algorithm uses this index to deterministically identify and process its specific slice of the search space, achieving massive throughput without the overhead of constant inter-node network communication.

Within those individual tasks, intra-node multiprocessing is often utilized to fully saturate the assigned CPU cores. However, launching multiple concurrent processes can lead to severe memory duplication, which is fatal for memory-intensive graph traversals. To mitigate this, concurrent architectures employ Inter-Process Communication (IPC) and shared memory. By utilizing thread-safe locks and shared variables, multiple worker processes can safely read global data structures and update shared states, such as a global incumbent bound, without triggering race conditions or redundant memory allocation.

2.4 Data Acquisition Concepts

To populate large-scale routing networks, real-world scheduling data must be extracted programmatically from remote servers. The standard mechanism for this is a REST API (Representational State Transfer Application Programming Interface). APIs provide structured endpoints where a client can send HTTP requests to retrieve specific, filtered datasets. In modern web architectures, this data is almost universally exchanged in JSON (JavaScript Object Notation), a lightweight, key-value serialization format. JSON allows nested, hierarchical data to be seamlessly transmitted over networks and deserialized directly into programmatic data structures, such as dictionaries and arrays, for immediate computational use.

However, when public APIs are unavailable or heavily restricted, data must be gathered via web scraping. Web scraping involves automating HTTP requests to target servers and parsing the resulting HTML, a markup language designed for human-readable browsers rather than machine consumption. This requires algorithmic extraction of target variables from unstructured or semi-structured Document Object Model (DOM) trees. Both API querying and web scraping are strictly governed by server-side rate limits (throttling). Consequently, data acquisition pipelines must implement pagination logic, exponential backoff strategies, and fault-tolerant parsing to ensure reliable, large-scale dataset generation without triggering server bans or connection timeouts.

3

Data Acquisition and Processing Pipeline

3.1 Sourcing Transit Data

The foundational step of this optimization problem was acquiring a comprehensive, time-accurate dataset of scheduled public transportation. The Guinness World Record guidelines mandate the use of scheduled public transport. Therefore, the data pipeline required a fusion of short-haul flight networks and high-speed rail schedules.

The geographic scope of the dataset was strictly and intentionally limited to the European continent. Europe is the only logical theater for this specific record due to its unparalleled density of sovereign states coupled with highly developed, cross-border transit infrastructure. However, extracting continental-scale transit data presents a significant financial barrier. Commercial aggregators typically charge per query, which is prohibitively expensive when building a dense, multi-country matrix. Consequently, a primary constraint of this project was to secure massive data volumes while strictly minimizing or eliminating acquisition costs. This budget constraint fundamentally dictated the technological approach for both flight and train data and imposed some challenges for acquiring high quality data.

3.1.1 Flight APIs

While the airline industry is highly standardized, access to global distribution systems (such as Amadeus or Sabre) requires expensive commercial licensing. To bypass these costs, the flight dataset was sourced using the AviationEdge API.

This platform offered a generous 1 month trial period charged at $7 that enabled access to most of the endpoints, among those the *flightsHistory* endpoint used to acquire the flight data. To efficiently target the API without exhausting query limits on irrelevant locations, an online global dataset of airports was acquired and programmatically filtered to isolate only active commercial airports within European borders. The resulting list of IATA (International Air Transport Association) codes served as the exact querying parameters.

By iterating through these European IATA codes and a specific 48-hour target date, the pipeline sent automated HTTP requests to extract all scheduled departures. The 48-hour window is required to ensure that routes starting at any point on day 1 have a full 24-hour window ahead of them to complete the record.

3.1.2 Train Data

Acquiring European rail data proved significantly more complex. Unlike aviation, the European rail network is heavily fragmented across dozens of national operators (e.g., SNCF, DB, Renfe, Trenitalia). There is no centralized, free REST API that aggregates all European train schedules; and master datasets are extremely expensive (often approaching five figures).

Because purchasing access to a unified commercial rail database was out of budget, the pipeline relied on programmatic web scraping. The target was a centralized rail search engine acting as a routing aggregator for a large portion of cross-border connections, namely the *Deutsche Bahn (DB) portal*.

Using the web scraping methodologies outlined in Chapter 2, a custom batch scraper was engineered to extract this data. Because modern transit portals heavily throttle automated requests, the script utilized *undetected-chromedriver* running in a headless Selenium environment to bypass bot-detection mechanisms. To guide the scraping process without blindly querying thousands of unviable city combinations, the pipeline leveraged an online compilation of major European rail corridors. The extraction algorithm systematically iterated through this predetermined list, simulating user queries between the station pairs in both forward and reverse directions. To ensure exhaustive daily coverage without triggering UI blocks, the scraper incrementally advanced the search time based on the last extracted departure, systematically sweeping the entire 24-hour window. Because scraping at this scale is computationally heavy and highly vulnerable to IP bans, the pipeline extracted exactly one standard weekday of train data. The algorithm operated under the safe assumption of schedule continuity, duplicating this 24-hour block to construct the necessary 48-hour window required for the record attempt.

The script then parsed the resulting unstructured HTML pages, isolated the Document Object Model (DOM) elements containing the departure, arrival, and transfer times, and serialized the extracted text into clean, incremental CSV records. This brute-force data engineering approach bypassed the lack of a unified API, successfully capturing the ground-level connectivity required for the routing algorithm.

3.2 Data Cleaning and Normalization

With the raw flight and train schedules acquired, the data existed in highly disparate, semi-structured formats across thousands of CSV and JSON files. To safely process and mutate this data at scale, a local SQLite database was instantiated as an intermediate staging environment. This approach enabled structured SQL queries to filter, group, and reconcile the massive datasets before exporting the finalized, clean records back to CSV format for the graph construction algorithm.

3.2.1 Entity Resolution and Standardization

A major challenge of aggregating multi-source European transit data is the lack of standardized naming conventions. A single physical location might appear under multiple aliases (e.g., “München Hbf”, “Munich Central”, or “München Hauptbahnhof”). If left uncorrected, the routing graph would treat these as distinct, disconnected nodes, destroying viable transfer opportunities.

To resolve this, a programmatic string normalization pipeline was implemented. This pipeline converted text to lowercase, stripped punctuation, removed generic tokens (e.g., “station”, “gare”, “stn”), and applied a hardcoded dictionary mapping to forcefully unify critical European hubs and border crossings under a single canonical name.

Additionally, although the flight API returned comprehensive historical delay metrics, these were ultimately stripped from the final dataset. Incorporating unpredictable historical delays into a time-expanded routing graph exponentially increases complexity without guaranteeing future accuracy. Consequently, the dataset was strictly normalized to rely solely on scheduled departure and arrival times to maintain a deterministic model.

3.2.2 The Midnight Rollover Problem and Absolute Minutes

To allow mathematical evaluation of layover times, all chronological data had to be converted into a continuous timeline. The target metric was *Absolute Minutes*, representing the elapsed time from $t = 0$ (midnight on the first day of the 48-hour routing window).
While flight data included explicit date strings, making absolute conversion straightforward, the scraped train data only provided localized *HH:MM* strings for intermediate stops. This introduced the *Midnight Rollover* problem: the data did not explicitly state when a train's journey crossed into the next calendar day.
To solve this, a chronological tracking algorithm was applied to the sequence of stops. By storing the absolute departure time of a route's origin, the algorithm deduced day boundaries by checking whether a subsequent stop's minute-value was numerically lower than the previous stop's value. If an arrival time was lower than its preceding departure time, 1440 minutes (24 hours) were automatically added to the timestamp, carrying the route into the next day. The algorithm also imputed missing values; if an arrival time was missing from the scraped HTML, it safely duplicated the departure time, ensuring no `NULL` values broke the final graph.

3.2.3 UTC Synchronization

Because the European continent spans multiple time zones (from UTC+0 in Portugal to UTC+2 in Finland), local transit times could not be directly compared. For example, a train departing Spain (UTC+1) at 10:00 and arriving in Portugal (UTC+0) at 09:30 would create a negative time-travel paradox if plotted in local time on a graph.
To ensure strict chronological consistency, every absolute minute timestamp was mapped to a universal UTC+0 baseline. This required building a comprehensive lookup table linking station locations to their respective timezone offsets. Since generic country-code matching is flawed for outermost regions, specific airport and station overrides were manually coded. For example, while mainland Spain is UTC+1, the Canary Islands were explicitly hardcoded to UTC+0. Similarly, vast countries like Russia required specific regional mapping. By subtracting these precise local offsets, the entire 48-hour European transit dataset was synchronized onto a single mathematically viable timeline.
Finally, a strict validation pass was executed across the normalized dataset. Any transit record where the absolute UTC arrival time was numerically lower than the absolute UTC departure time was permanently dropped, effectively filtering out upstream API scheduling errors and corrupted data points.

3.3 Spatial Mapping and Filtering

To evaluate a Guinness World Record attempt based on the number of countries visited, the routing algorithm must inherently "know" the geographical location and sovereign territory of every single node in the network. Furthermore, a raw European transit graph contains tens of thousands of minor stops (e.g., suburban commuter stations or rural villages) that drastically bloat the search space without offering viable international transfer opportunities. Consequently, a rigorous spatial mapping and filtering pipeline was engineered to reduce the graph to its most mathematically relevant hubs while ensuring 100% geographical accuracy.

3.3.1 Country Assignment and Border Resolution

While international flights explicitly land in known countries, domestic and cross-border train schedules scraped from the DB portal do not consistently label the country of intermediate stops. To dynamically resolve this, a multi-strategy geographical detection algorithm was implemented.
The algorithm evaluated each scraped station name through a descending hierarchy of spatial heuristics:

- **Regex code extraction:** Extracting explicit country codes often hidden in parentheses within the raw text (e.g., parsing (CH) as Switzerland or (NL) as the Netherlands).

- **Language pattern recognition:** Utilizing linguistic markers to confidently infer sovereign territory (e.g., the presence of "hl.n." for Czech main stations, "Glowny" for Poland, or " b.Wien" for Austrian suburbs).
- **GeoNames cross-referencing:** Querying a localized subset of the GeoNames database to map normalized station strings to the most populous European city matching that name.

Despite these programmatic layers, complex border stations frequently generated false positives (e.g., "Genève" being administratively in Switzerland despite a French linguistic profile, or border towns like Domodossola). To guarantee absolute integrity for record evaluation, an exhaustive list of manual topological corrections was hardcoded into the pipeline to forcefully overwrite erroneous geographical assignments at critical border crossings.

3.3.2 Station Filtering and Hub Identification

With countries successfully assigned, the sheer volume of total nodes remained computationally intractable. A scoring heuristic was developed to prune the network while safely preserving the core international skeleton. Every station was evaluated based on keyword presence (e.g., rewarding "Hauptbahnhof" or "Central", penalizing "Village" or "Halt") and its topological frequency across all scraped routes.

The filtering logic dictated that the first and last stops of any train were always preserved, alongside at least one high-scoring hub per traversed country. This successfully stripped away thousands of irrelevant intermediate stops without breaking the physical continuity of the routes.

To further isolate the absolute most critical transit hubs among the roughly 40 targeted countries, the pipeline leveraged Large Language Models (LLMs). By feeding national station lists into an advanced LLM, the model was instructed to intelligently identify only the most viable international transfer hubs for each specific country, keeping the final graph dense with high-value opportunities.

3.3.3 Intra-Country Transfers and Geocoding Validation

The final geographical challenge was enabling realistic human transfers between different stations within the same city (e.g., moving from a flight arriving at Paris CDG to a train departing from Paris Gare du Nord). Establishing a physical connection between two nodes requires highly precise geospatial coordinates and transit times.

Because commercial geocoding and routing APIs (such as the *Google Maps Distance Matrix API*) incur significant financial costs at scale, an innovative, multi-agent AI approach was designed. An LLM was prompted to evaluate the filtered hubs and identify which intra-city stations were actually worth connecting. A strict constraint was applied: any transfer requiring excessive transit time (e.g., 4 hours) was automatically discarded, as such a delay would mathematically destroy a 24-hour record attempt.

This process generated a highly focused list of 236 viable transfer connections, alongside AI-estimated coordinates and durations. Because LLMs are prone to geospatial hallucinations, a secondary LLM was deployed specifically to audit the first model's output. However, to eliminate any margin of error and ensure maximum academic rigor, a manual validation protocol was executed. Every single one of the 236 critical transfers was manually queried in the Google Maps UI to extract the exact walking or local-transit durations. This painstaking manual verification completely eliminated API costs while yielding a 100% accurate, human-viable transfer matrix for the final routing algorithm.

4

Algorithmic Approach I: Label Setting and Bounding Foundations

4.1 The Time-Dependent Transportation Graph

With a fully normalized, UTC-synchronized dataset of European transit schedules, the next step was constructing the mathematical environment for the routing algorithm. A standard static spatial graph $G = (V, E)$ is fundamentally inadequate for modeling scheduled public transportation, as an edge implies continuous availability.

To rigorously test different modeling paradigms, the first architectural approach evaluates the network as a time-dependent multi-graph. In this Activity-on-Edge (AoE) architecture, the vertices remain purely spatial, while the dimension of time is encoded directly into the attributes of the edges. This approach was deliberately chosen to maintain a highly compact memory footprint, avoiding the node explosion inherent to fully time-expanded models.

The graph was instantiated using the NetworkX library with the following structural rules:

- **Spatial Nodes (V):** A single vertex was created for each unique physical location. Nodes were tagged with specific attributes: their sovereign country (critical for the objective function) and their infrastructure type (airport or station).
- **Temporal Transit Edges (E):** Flights and trains were modeled as directed edges between origin and destination nodes. Because multiple trains or flights travel between the same two cities throughout the day, these edges stored dynamic temporal attributes. Instead of a single static weight, each edge contained an array of valid $(departure\_time, arrival\_time)$ pairs, formatted in absolute UTC minutes. To account for the 48-hour Guinness World Record window, train schedules (which operate on daily cycles) were programmatically duplicated, adding 1440 minutes to populate the second day of the graph.
- **Static Transfer Edges:** The manually validated intra-city connections (e.g., traversing from a city's airport to its central rail station) were integrated as standard, bidirectional static edges. Unlike transit edges, transfer edges were assigned a fixed scalar weight representing the human connection time in minutes, as they can be traversed at any point during the day.

By compressing schedules into arrays on spatial edges, this AoE architecture successfully avoided node redundancy. However, this time-dependent structure shifts the chronological burden entirely onto the search algorithm, which must dynamically iterate through edge arrays to find valid connections based

on its arrival time at any given node. This compact graph serves as the baseline environment to test the exact optimization methods detailed in the following sections.

4.2 Algorithm Evolution: From Baseline to Bounding

Because the Guinness World Record challenge is fundamentally a multi-objective optimization problem (maximizing unique countries visited while minimizing time within a strict 24-hour limit) a standard shortest-path algorithm like Dijkstra's cannot be used. Instead, the search must be modeled using Dynamic Programming (DP) via a Label-Setting algorithm (see Algorithm 1 in Appendix X).
In this paradigm, as the algorithm traverses the time-dependent multi-graph, it assigns a state "label" L to each node. A label is defined by the tuple $L = (v, t, C, start\_time)$. Here, v tracks the current node, t is the current absolute minute, C represents the mathematical set of unique countries visited so far, and $start\_time$ marks when the journey began. The cardinality of the set, denoted as $|C|$, represents the integer count of unique countries visited.
To prevent the search space from expanding infinitely, the algorithm relies on Pareto Dominance. At any given node, a new label L' is strictly dominated by an existing label L'' if the existing path arrived earlier or at the same time ($L''.t \leq L'.t$) while having visited a superset or equal set of countries ($L''.C \supseteq L'.C$), with at least one condition being strictly better. Dominated labels are instantly discarded.
However, implementing this baseline algorithm revealed a fatal flaw: the Pareto dominance conditions were too relaxed. Across a network spanning roughly 40 countries, the combinatorial permutations of visited country sets C caused an explosion of non-dominated labels, rendering the baseline approach computationally unfeasible. This initiated a rigorous, iterative evolution of the algorithm, where advanced mathematical pruning techniques were progressively introduced to control the state space.

4.2.1 Reachability and Upper Bound (UB) Pruning

The first major architectural change was introducing predictive pruning (see Algorithm 2 in Appendix X). Before expanding a label, the algorithm calculated an optimistic Upper Bound (UB) of the maximum new countries that could theoretically be reached from the current node v within the remaining time.
To make this computationally tractable without running a full sub-search at every node, time was discretized into fixed intervals, or "buckets" b. For any given node v and time bucket b, a precomputation step generated $R(v, b)$, the absolute set of all sovereign countries reachable from that location before the 24-hour clock expired. The theoretical Upper Bound was then calculated as the cardinality of the set difference between the reachable countries and the already visited countries: $UB = |R(v, b) \setminus C|$.
If the current number of countries visited plus this theoretical maximum ($|C| + UB$) was strictly less than or equal to the current global record, the path was mathematically dead and immediately pruned.

4.2.2 Time-Budgeted UB and Minimum Transit Time (MTT)

While standard UB pruning successfully removed hopeless paths, it was overly optimistic. It assumed that if 10 new countries were reachable from a node, all 10 could be visited, completely ignoring the physical time required to travel between them.
To tighten this bound, the concept of Minimum Transit Time (MTT) was introduced. The MTT represents the absolute shortest physical time required to enter, traverse, and exit a specific country's transit network. For instance, even under perfect schedule conditions, traveling across Switzerland or Germany requires a mathematically provable minimum number of minutes. The algorithm precomputed this MTT for every country in the dataset.
When calculating the UB, the algorithm sorted all unvisited, reachable countries from the fastest MTT to the slowest (see Algorithm 6 in Appendix X). It then sequentially added these minimum transit times to a `time_spent` counter. The moment the accumulated `time_spent` exceeded the actual hours remaining in the 24-hour journey, the algorithm stopped incrementing the UB. This "Time-Budgeted UB"

drastically accelerated convergence: it accurately pruned paths that possessed theoretical geographic connectivity but lacked the physical time budget required to actually execute the transits.

4.2.3 Priority Scoring, Hard Floors, and Rollouts

To further optimize the search, the standard queue was replaced with a Priority Queue. Rather than expanding blindly, labels were ranked using a custom scoring formula. The formula was designed to prioritize deep, fast routes. For example, a core heuristic evaluated $Score(L) = |C| \times 10000 - t$, effectively forcing the algorithm to aggressively expand labels with the highest country count, while using an earlier absolute time t as the mathematical tie-breaker to favor faster connections (see Algorithm 4 in Appendix X).

Despite prioritized expansion, execution logs revealed the algorithm still wasted resources expanding labels near the end of the timeline that had only accumulated a few countries. To counter this, a Global Hard Floor was introduced (see Algorithm 9 in Appendix X). By establishing a GlobalMinTime (the minimum realistic time to visit any single country), the algorithm calculated the physical time required to bridge the gap between $|C|$ and the target record. If this required time exceeded the remaining clock, the label was killed. Crucially, all arbitrary parameters for these heuristics (such as setting the GlobalMinTime to just 25 minutes) were deliberately defined to be overly optimistic. The architectural philosophy was to tolerate redundant computational overhead rather than risk pruning a mathematically viable, record-breaking route.

To capitalize on the Priority Queue, a Monte-Carlo Rollout mechanism was injected (see Algorithm 8 in Appendix X). Every 1,000 iterations, the algorithm paused standard expansion and launched a rapid, randomized "smart-greedy" probe deep into the graph (see Algorithm 7 in Appendix X). Using heavily biased weights to favor unvisited countries and a Tabu list to prevent local loops, these rollouts operated in milliseconds. If a rollout successfully reached the end of the day and beat the current global record, the new record was dynamically adopted, triggering a massive, immediate pruning wave across the entire Priority Queue.

4.2.4 The Bi-directional Search Pivot

The final algorithmic evolution within this paradigm was born from observing the failure points of the Monte-Carlo rollouts. Debugging revealed that rollouts were highly ineffective when they reached the night hours; physical train schedules became incredibly sparse, trapping the greedy probes in geographic dead-ends.

To bypass this nocturnal sparsity, the monolithic 24-hour search was torn in half. The architecture was rewritten as a Bi-directional Search: one algorithm ran forward in time from the start of the day, while an identical algorithm ran backward in time from the end of the day (see Algorithms 10 and 11 in Appendix X). Both halves executed for 14 hours, creating a 4-hour overlap in the middle of the day.

Finally, a dedicated "Stitcher" algorithm evaluated the intersection of these two sets (see Algorithm 12 in Appendix X). By iterating through overlapping nodes and validating time continuity and geographic unions ($C_{forward} \cup C_{backward}$), the stitcher could reconstruct a globally optimal 24-hour path. This parallel, bi-directional approach effectively bypassed the sparse night-time bottlenecks and maximized graph coverage during the dense daytime schedules.

4.3 The Dimensionality Collapse

Despite the rigorous evolution of the Label-Setting algorithm, from baseline Pareto dominance to time-budgeted Upper Bounds, hard floors, and bi-directional stitching, the architecture ultimately succumbed to the combinatorial complexity of the European transit network. It is crucial to clarify that the algorithm did not fail due to an Out-Of-Memory (OOM) error. Rather, it hit a computational "time wall," suffering a catastrophic degradation in execution speed.

The root of this bottleneck lay in the maintenance of the Pareto frontiers. As the search progressed deeper into the time-dependent multi-graph, the frontiers at each node grew increasingly dense. Because the Guinness World Record routing is fundamentally multi-objective (minimizing time versus maximizing unique countries), vast numbers of paths are mathematically incomparable. A route that visits 8 countries by 14:00 cannot dominate a route that visits 9 countries by 16:00. Consequently, both labels must be kept alive. As these non-dominated sets grew exponentially, the algorithm was forced to perform extensive comparative loops for every newly generated label just to establish dominance, rendering the queue extraction and expansion process computationally intractable.

This inefficiency was heavily compounded by the execution environment. The algorithm was executed locally on a single machine rather than being deployed to a High-Performance Computing (HPC) cluster. This was not a limitation of resources, but a deliberate architectural necessity. Dynamic Programming and Label-Setting algorithms fundamentally rely on continually evaluating and updating a globally shared state, specifically, the dynamic Pareto frontiers at every node in the graph. Attempting to parallelize this specific architecture across distributed cluster nodes (which operate on isolated memory pools) would have introduced massive Inter-Process Communication (IPC) overhead. The constant network latency required to safely lock and synchronize the frontiers across dozens of different parallel workers would have easily neutralized, if not worsened, the computational time. Thus, the approach was inherently bottlenecked by single-thread CPU execution speeds.

After weeks of continuous local execution, the algorithm struggled to converge on any record-breaking paths,a stark contrast to the millions of valid routes that would later be discovered by subsequent methods. It is worth acknowledging that performance could theoretically have been pushed further. By implementing stricter, non-optimistic heuristics (such as aggressively killing technically viable paths to artificially force the queue to empty faster), the algorithm could have achieved faster convergence. However, doing so would have completely sacrificed the exactness of the search. Given the highly specific, non-intuitive nature of train schedules, overly aggressive pruning risked the blind deletion of the optimal, record-winning route.

Having thoroughly tested the boundaries of the Time-Dependent Activity-on-Edge (AoE) graph, the first phase of this comparative study was concluded. The execution bottlenecks clearly demonstrated that while the AoE model achieves exceptional memory efficiency, it imposes an unsustainable temporal and comparative burden directly on the search algorithm. With the exact computational trade-offs of this paradigm established, the analysis now transitions to evaluating the second architectural candidate: the highly parallelizable Activity-on-Node (AoN) framework explored in the following chapter.

5

Algorithmic Approach II: Graph-Based DFS Exploration

5.1 The Activity-on-Node (AoN) Transformation

As established in the preliminary definitions, a central objective of this research is to evaluate and contrast two fundamental graph paradigms for time-dependent routing. Having quantified the computational limits of the memory-efficient Activity-on-Edge (AoE) architecture in Chapter 4, the analysis now shifts to the second structural candidate: the Activity-on-Node (AoN) framework. The AoE model prioritized a compact spatial topology, which inherently forced the search algorithm to dynamically calculate temporal validity at every node. To systematically test the inverse trade-off, the European transportation network was mathematically reconstructed as an Activity-on-Node Directed Acyclic Graph (DAG) (see Appendix X). This structural pivot deliberately transfers the chronological burden away from the search algorithm and embeds it entirely into the static topology of the graph, prioritizing algorithmic traversal speed over memory footprint.

5.1.1 Structural Inversion: Events as Nodes

In the AoN paradigm, physical locations no longer exist as nodes. Instead, every single scheduled transport event (a specific flight or train) is instantiated as an independent vertex. A node in this graph is defined by the rigid tuple $v = (\text{Origin}, \text{Destination}, \text{Departure\_Time}, \text{Arrival\_Time})$. For example, a flight leaving Barcelona at 10:00 and arriving in Rome at 12:00 represents one single, immutable node in the graph. By unpacking the temporal arrays from the original 8,456 spatial edges, the AoN transformation generated exactly 49,982 distinct event nodes (see Appendix X).

Consequently, the edges in the AoN graph no longer represent physical travel. Instead, directed edges represent valid transfer opportunities between two events. A directed edge from event node A to event node B is drawn if and only if:

- Event A's destination matches Event B's origin (or they are connected via a valid intra-city transfer).
- Event A's arrival time strictly precedes Event B's departure time, fully accounting for minimum physical transfer durations (e.g., $\text{Arrival}_A + \text{Transfer\_Time} \leq \text{Departure}_B$).

5.1.2 Trade-offs and the Directed Acyclic Guarantee

This structural inversion offered a massive computational advantage: the search space became entirely static. Because every edge in the AoN graph guarantees chronological validity, a Depth-First Search (DFS) algorithm can traverse the network blindly. It never has to pause to calculate if a connection is mathematically legal; if an edge exists, the transfer is guaranteed to be valid. Furthermore, because time strictly moves forward, the resulting graph is a Directed Acyclic Graph (DAG). It is physically impossible to traverse a loop, which naturally prevents infinite cycles without requiring complex Tabu lists.

However, this advantage comes with a severe structural trade-off: an exponential explosion in edge density. In a densely connected continental network, a single train arriving at a major hub like Frankfurt might theoretically connect to hundreds of subsequent departing trains. If every valid transfer opportunity is explicitly drawn as an edge, the branching factor of the DAG becomes completely unmanageable for a DFS traversal.

5.1.3 Earliest Arrival Pruning

To tame this exponential branching factor, a highly aggressive, domain-specific heuristic was engineered during the graph construction phase: Earliest Arrival Pruning (see Appendix X).

When evaluating all valid subsequent events (node B candidates) from a given arrival event (node A), the transformation algorithm grouped the candidates by their final destination. Rather than drawing edges to all valid subsequent trains heading to a specific city, the algorithm only drew a single edge to the train that offered the absolute earliest arrival time at that destination.

Mathematically, if Event A arrives in Munich at 14:00, and there are three valid subsequent trains to Vienna departing at 14:30, 15:00, and 16:00, creating edges to all three is redundant for a time-minimization objective. The 14:30 train dominates the others. By rigidly enforcing this pruning rule, tens of thousands of redundant temporal edges were mathematically severed before the search even began. Validation scripts confirmed this heuristic successfully reduced the total edge volume by over 75%, transforming an impossibly dense theoretical web into a streamlined, highly traversable DAG containing approximately 1.5 million strictly optimal transfer edges (see Appendix X).

5.2 Baseline DFS Execution on the Full Graph

With the European transit network successfully transformed into an Activity-on-Node (AoN) Directed Acyclic Graph (DAG) and stripped of redundant temporal edges, the search space was structurally primed for a pure Depth-First Search (DFS). Unlike the Dynamic Programming approach evaluated in Chapter 4, which required maintaining continuously expanding Pareto frontiers in memory at every node, a DFS on a DAG only requires maintaining the current path stack. This exceptionally low memory footprint eradicated the previous state-space memory bottlenecks, allowing the algorithm to trace individual routes to completion with virtually zero memory overhead.

5.2.1 Core Search Mechanics and Bounding

At its conceptual core, the DFS algorithm is highly intuitive: it selects a valid starting node (e.g., a specific flight departing at 08:00) and exhaustively explores every possible sequence of connecting transports. The algorithm "walks" down a continuous path of flights and trains, accumulating countries along the way, until the 24-hour time limit expires. Once a path hits the deadline, the algorithm evaluates the total number of unique countries visited. If the total beats or ties the current global record, the route is saved. The algorithm then takes a step back (backtracks) to the previous station and tries the next available connecting branch, systematically exploring every single branch of the tree.

However, even on a highly pruned AoN graph, a pure brute-force traversal of a continental network would take centuries to execute. To control the exponential branching factor, the algorithm took direct

advantage of the mathematical bounding techniques developed in Chapter 4. The Minimum Travel Time (MTT) and Upper Bound (UB) logic were seamlessly ported into the DFS as a Branch and Bound (B&B) mechanism. At every step of the traversal, the algorithm calculated the time remaining before the 24-hour deadline, divided it by the MTT of the current country, and added this theoretical maximum to the current country count. If this optimistic Upper Bound could not mathematically beat the highest global record found thus far, the algorithm immediately abandoned that path and backtracked, saving millions of useless computational cycles.

5.2.2 High-Performance Multiprocessing Architecture

Because a DFS evaluates independent branches, the search is an "embarrassingly parallel" problem. The architecture capitalized on this by gathering all valid starting nodes in the graph, defined as any transit event arriving early enough to allow a full 24-hour window, and distributing them across a pool of independent worker processes. Each worker was assigned a specific starting node and made solely responsible for exhaustively traversing all possible paths originating from that event.
To execute this at scale, the algorithm was deployed to a High-Performance Computing (HPC) cluster using the SLURM workload manager, allocating a single, highly powerful compute node equipped with 68 physical CPU cores and 64 GB of RAM.
Parallelizing a massive graph search in Python, however, introduces severe serialization and Inter-Process Communication (IPC) overheads. To prevent these systems-level bottlenecks from crippling the search speed, two critical engineering optimizations were implemented:

- **Zero-Serialization Memory Sharing:** Passing a 1.5 million-edge graph to 68 individual worker processes via standard Python multiprocessing would crash the system due to memory duplication overhead. Instead, the architecture utilized an initialization function (`init_worker`). This allowed the global graph dictionary, country mappings, and MTT data to be loaded natively into each worker's local memory space upon instantiation, completely bypassing continuous serialization.

- **Lazy Synchronization:** To allow all 68 workers to globally prune branches, they needed to share a single `shared_global_max` variable representing the current highest country record. Forcing 68 cores to constantly acquire a multiprocessing lock to read this variable would cause massive IPC contention. To solve this, a "lazy sync" architecture was designed: each worker maintained a local copy of the global maximum and only paid the IPC penalty to synchronize with the shared manager once every 10,000 nodes explored, or when a new record was actively discovered.

5.2.3 Live Checkpointing and the 48-Hour Bottleneck

To safeguard against cluster node failures or strict execution time limits, a fault-tolerant live-checkpointing system was integrated. As the 68 cores independently plunged into the graph, any path that successfully accumulated 14 or more countries, or tied the global maximum, was instantly serialized and appended to a `live_checkpoint.json` file. This ensured that valuable, highly optimized candidate routes were continuously preserved to disk during execution without waiting for the entire algorithm to finish.
The algorithm was submitted to the cluster with a maximum wall-clock execution limit of 48 hours. Over the course of the run, the HPC node operated at maximum capacity, exploring billions of nodes and aggressively pruning unpromising branches.
However, when the 48-hour hard limit was reached and the cluster terminated the job, the execution logs revealed a humbling reality. Despite the extreme memory efficiency of the AoN DFS, the strict Branch and Bound pruning inherited from Chapter 4, and the deployment of 68 parallel cores, the algorithm had evaluated barely 1% of the total starting nodes in the graph. While the AoN transformation had successfully solved the memory collapse of the previous paradigm, the unconstrained branching factor of the full, unfiltered continental network still presented an insurmountable time complexity barrier for exhaustive evaluation.

5.3 Graph Reduction and Targeted Executions

Although the 48-hour baseline DFS execution only evaluated 1% of the starting nodes in the full AoN graph, the live-checkpointing architecture yielded an extraordinary dataset. When the execution was terminated, the checkpoint file contained over 100,000 highly optimized, record-breaking candidate routes. This sheer volume of successful paths, extracted from such a tiny fraction of the search space, suggested a critical topological hypothesis: while the algorithm was suffocating under the combinatorial weight of the full continental network, the actual winning routes were likely relying on the exact same core infrastructure.

To empirically test this hypothesis, a custom analytical dashboard was developed to parse the 100,000+ routes generated by the baseline run. The dashboard performed a rigorous Pareto analysis (frequency vs. cumulative percentage) on station and edge usage. The visual and statistical outputs conclusively confirmed the hypothesis: a small, elite subset of major international hubs and high-speed corridors was handling the vast majority of the routing traffic. Conversely, the dashboard's "long-tail" analysis revealed thousands of regional stations that appeared in almost zero winning permutations. The baseline algorithm had been spending the bulk of its 48-hour execution window exhaustively calculating transfers between minor suburban towns that possessed no mathematical viability for a continental speed record.

5.3.1 Data-Driven Graph Reduction

Armed with this empirical evidence, the focus shifted from an exhaustive continental search to a targeted, high-density graph reduction. Rather than arbitrarily guessing which cities were important, the reduction was strictly data-driven. A pipeline was engineered to ingest the live checkpoint data, calculate the exact appearance frequency of every station across the 100,000 winning routes, and dynamically sever the low-value regional branches.

Using the original spatial graph as a baseline, two specific reduced subgraphs were generated:

- **The `freq100` graph (conservative):** A subgraph retaining only stations that appeared more than 100 times in the elite checkpoint routes.
- **The `freq1000` graph (aggressive):** A highly concentrated subgraph retaining only the absolute most critical arterial stations, requiring more than 1,000 appearances in the elite routes.

This programmatic pruning successfully stripped away the dense, localized noise, leaving only the high-speed, cross-border infrastructure that possessed proven viability. These reduced spatial graphs were then transformed into new, significantly lighter AoN DAGs.

5.3.2 Full Convergence on Reduced Graphs

The identical 68-core DFS algorithm was subsequently deployed on these reduced AoN graphs. By mathematically severing the low-value regional branches, the algorithm was forced to strictly traverse the high-speed arterial core. The reduction in the branching factor yielded breathtaking computational improvements.

Whereas the full graph had timed out after 48 hours while exploring just 1% of the space, the algorithm running on the aggressively pruned `freq1000` graph reached absolute completion, exhaustively evaluating 100% of all possible starting nodes and paths, in just 30 minutes. The conservatively pruned `freq100` graph, which retained a wider geographic net, successfully ran to absolute completion in exactly 3 hours.

By intelligently leveraging the partial outputs of a failed brute-force attempt, the pipeline successfully bypassed the combinatorial explosion. The DFS was finally able to evaluate the entirety of the relevant temporal network, guaranteeing the extraction of the absolute mathematical global optimums for the Guinness World Record attempt.

5.4 Distributed DFS Execution

Following the baseline DFS execution, logs indicated that the algorithm had evaluated only 1% of the valid starting nodes before hitting the 48-hour wall-clock limit. However, the nature of the Activity-on-Node (AoN) graph presented an opportunity: because each starting node represents an independent search tree, the problem is embarrassingly parallel. The mathematical deduction was straightforward: if a single compute node could evaluate 1% of the search space in 48 hours, distributing the workload across 100 compute nodes should theoretically allow for the exhaustive evaluation of the entire graph within the same 48-hour window.
To execute this, the architecture was upgraded from a single-node multiprocessing script to a fully distributed SLURM Job Array.

5.4.1 Horizontal Scaling and Job Array Distribution

The starting nodes were extracted, sorted deterministically, and partitioned across 100 independent array tasks. Rather than slicing the nodes into contiguous blocks, a round-robin distribution was implemented using modulo arithmetic (`index % total_jobs == job_idx`). This ensured that dense, high-traffic periods (e.g., morning rush hours) and sparse periods were evenly distributed across all 100 workers, preventing individual nodes from being disproportionately bottlenecked by heavy temporal branches.
To further optimize the local traversal within each worker, a State Dominance Memoization technique was introduced. Each worker maintained a local hash map of previously visited states, defined by the tuple $(\text{current\_node}, \text{visited\_countries})$. If a worker arrived at a state it had already reached via an earlier or identical arrival time, the branch was instantly pruned.

5.4.2 OOM Hurdles and Distributed Checkpointing

Deploying 100 parallel jobs, each running multiple worker processes and maintaining local memoization dictionaries, placed an immense strain on the cluster's memory infrastructure. Early deployments of the distributed array encountered severe Out-Of-Memory (OOM) errors, as the `local_visited_states` dictionaries ballooned in RAM. To stabilize the array, the memory allocation per task had to be significantly increased (up to 64 GB per node) and the core count per task carefully balanced to ensure workers did not cannibalize shared memory.
Additionally, having 100 independent jobs concurrently attempting to write to a single, shared JSON checkpoint file would have caused catastrophic I/O contention and file corruption. Instead, the checkpointing architecture was decentralized. Each job array task was assigned a unique, append-only JSONL (JSON Lines) file. As workers discovered routes visiting 14 or more countries, the paths were serialized as individual JSON objects and safely appended to their respective files, eliminating all locking contention.

5.4.3 Route Deduplication and Parquet Compilation

Once the 100-node array completed its execution, the output consisted of 100 disparate JSONL files containing hundreds of thousands of candidate routes. Because the European transit network contains multiple converging paths, different starting nodes frequently converged onto the exact same optimal sequence of subsequent trains, resulting in massive data duplication across the array.
To synthesize this fragmented output, a dedicated compilation pipeline was engineered. The compiler ingested every JSONL file and generated a deterministic MD5 hash for each route based on its sequence of stations. This hashing mechanism successfully identified and dropped millions of redundant duplicate routes.
Finally, the deduplicated elite routes were flattened into a tabular structure and exported as a highly compressed `.parquet` file. This transition from nested JSON to columnar Parquet drastically reduced

the dataset's footprint and unlocked the ability to perform high-speed analytical queries. The resulting database contained the definitive "master list" of elite 14+ country routes, setting the stage for the data-driven analysis and graph reduction that followed.

5.5 Route Quality and Algorithmic Blind Spots

The distributed DFS successfully generated millions of candidate routes visiting 14 or more countries. However, raw mathematical validity does not perfectly align with human feasibility. Because the algorithm was strictly parameterized to maximize country count within 24 hours, it ruthlessly exploited the mathematical limits of the graph. For example, the algorithm frequently selected "impossible" transfers, such as an itinerary arriving at a major airport and boarding a subsequent international flight departing just two minutes later. While mathematically valid within the strict confines of the AoN DAG, such a transfer is physically impossible for a human runner needing to traverse terminals, pass security, or manage gate closures.

5.5.1 The Route Quality Index (RQI)

To bridge the gap between mathematical optimality and physical reality, all candidate routes were subjected to a post-execution evaluation using a custom Route Quality Index (RQI). The RQI dynamically scores the physical robustness of a route by evaluating the feasibility of every individual transfer, accounting for specific transportation modalities (airport "A" vs. rail/bus "R").
For each transfer, the engine calculates the available safety margin. The `raw_buffer` is the time between arrival and the next departure. From this, the algorithm subtracts the physical intra-city `transit_time` (pulled from the graph, or defaulting to 60 minutes if moving between different unmapped stations) and a modality-specific `warrior_minimum`. These minimums reflect the absolute fastest human transit times: 20 minutes for A→A and A→R, 25 minutes for R→A (accounting for airport security), and 5 minutes for R→R (0 minutes if staying at the exact same rail station).
The effective safety margin is defined as

$$\text{Margin} = \text{Raw Buffer} - \text{Transit Time} - \text{Warrior Minimum}. \tag{5.5.1}$$

Each transfer receives a piecewise quality score q based on this margin:

- **Non-negative margin (Margin ≥ 0):** Scored on an asymptotic exponential curve capped at 100:

$$q = 100\big(1 - e^{-0.05 \cdot \text{Margin}}\big). \tag{5.5.2}$$

  A 30-minute safety buffer yields an excellent score (≈ 77), while anything above 90 minutes approaches 100.

- **Negative margin (Margin < 0):** Heavily penalized with a linear multiplier:

$$q = \text{Margin} \times 1000. \tag{5.5.3}$$

  A "miracle" connection missing the minimum by just 5 minutes receives a score of -5000.

Finally, because a single missed connection ruins the entire 24-hour attempt, the total RQI is calculated using a weighted formula that aggressively penalizes the route's weakest link (the bottleneck):

$$\text{RQI} = 0.7\, q_{\text{bottleneck}} + 0.3\, q_{\text{average}}. \tag{5.5.4}$$

By applying this formula, the dashboard successfully filtered out physically absurd routes, allowing the generated itineraries to be ranked strictly by their real-world viability.

5.5.2 The Missing Routes Paradox

During the cross-evaluation of the RQI-ranked routes, a significant mathematical anomaly was discovered. When comparing the output of the conservatively pruned `freq100` graph against the output of the massive 100-node distributed array running on the full graph, several highly optimized, high-RQI routes present in the `freq100` dataset were entirely missing from the distributed dataset.

Logically, this should be impossible. The `freq100` graph is a strict subset of the full graph; therefore, any route found in the subset must mathematically exist in the superset. Since the distributed array was designed to exhaustively explore the full graph without missing any nodes, it should have theoretically captured every single route found by the `freq100` run, plus thousands more.

The investigation into this paradox revealed a critical blind spot in the interaction between the State Dominance Memoization pruning and the chronological structure of the full graph. In the distributed worker script, a state was defined as $(\text{current\_node}, \text{visited\_countries})$. If the worker reached this state at an arrival time equal to or later than a previously recorded instance, the branch was heavily pruned to save computational time.

However, because the algorithm was blindly iterating through the full graph's massive branching factor, it frequently reached these states via mathematically "fast" but physically absurd connections (e.g., 2-minute airport transfers). The worker would record this impossibly fast arrival time as the "best known state." Later, when the DFS naturally evaluated the more realistic, human-viable path (e.g., a path with a safe 90-minute layover), the algorithm would see that the arrival time was slower than the previously recorded "miracle" route. Consequently, the memoization logic ruthlessly pruned the realistic route, preserving only the physically impossible path.

5.5.3 Heuristic Re-Engineering and Stack Sorting

The root cause of the Missing Routes Paradox lay in the State Dominance pruning logic. The distributed workers tracked the best arrival time for any given state, `local_visited_states[(node, visited_countries)] = best_known_arrival`. Because the DFS explored branches blindly, it often stumbled upon physically impossible 2-minute transfers first. It would record this absurdly early arrival time as the benchmark. Later, when the DFS evaluated a realistic 90-minute transfer reaching the exact same state, the arrival time was naturally later, triggering the strict pruning condition (`succ_arr >= best_known_arr`) and permanently deleting the viable route.

To counteract this, the search logic required a fundamental re-engineering to force the DFS to explore realistic layovers before miracle layovers. To achieve this without altering the integrity of the exhaustive search, a Heuristic Sorting mechanism was injected directly into the graph dictionary initialization.

During graph construction, the algorithm evaluated all valid successors for a given node. The ideal human layover was defined as 90 minutes. The successors were sorted in descending order (`reverse=True`) using an absolute-difference key:

$$\text{Sort Key} = \left|(\text{Departure}_{\text{next}} - \text{Arrival}_{\text{current}}) - 90\right|. \tag{5.5.5}$$

Because the DFS relies on a stack data structure, it operates on a Last-In, First-Out (LIFO) basis. This ordering had a direct mechanical effect:

- **Miracle and excessive transfers:** Layovers with extreme deviations from 90 minutes (e.g., a 2-minute transfer yielding a key of 88) were placed at the very front of the successor list, pushing them to the bottom of the DFS stack.
- **Optimal transfers:** Layovers close to 90 minutes (key near 0) were placed at the very end of the successor list, so they sat at the top of the DFS stack.

When the worker process called `stack.pop()`, it immediately extracted and explored the highly viable, $\sim$90-minute layovers first. This heuristic ordering forced the DFS to discover high-RQI routes

early, setting realistic benchmark arrival times in the memoization dictionary and helping high-quality itineraries survive pruning.

Aquesta ultima part esta corrent (el cluster em te bloquejant espero fins 1 d'abril, reescriure quan tingui resultats)

6 Results and Conclusions

6.1 Computational Results and Execution History

The primary objective of this research, to computationally evaluate the topological limits of the European transit network and extract the global optimal routing sequence, was successfully achieved. Solving the problem required navigating extreme computational bottlenecks, ultimately resulting in the deployment of five distinct search strategies. By analyzing the output of these successive iterations, the theoretical and feasible boundaries of the continent's transit infrastructure can be definitively established.

6.1.1 The Evolution of the Search Architecture

To conquer the state-space explosion inherent in the Activity-on-Node (AoN) graph, the routing engine evolved through five major computational phases:

- **Baseline DFS (full graph):** A single-node multiprocessing approach. Due to the unconstrained branching factor, this exhaustive search evaluated merely 1% of the starting nodes before triggering a 48-hour execution timeout.
- **Topological reduction (`freq1000`):** A heavily pruned subgraph retaining only the absolute elite international hubs (those appearing in at least 1000 routes from the Baseline DFS results). This search completed exhaustively in under an hour but lacked the secondary transit lines necessary to push the theoretical boundaries.
- **Topological reduction (`freq100`):** A moderately reduced subgraph (nodes appearing in at least 100 routes from the Baseline DFS results). This search completed exhaustively in three hours, capturing a significant volume of viable paths but arbitrarily excluding lesser-known functional transit events.
- **Distributed DFS (full graph, non-heuristic):** A 100-node High-Performance Computing (HPC) array deployed across the full unreduced network. While it achieved 100% exploration, its blind chronological traversal caused the local state-dominance memoization to overwrite realistic routes with mathematically fast, physically impossible "miracle" connections.
- **Distributed heuristic DFS (full graph):** The final architecture. By mathematically sorting successors to prioritize ideal 90-minute human layovers, this array achieved 100% exploration while protecting high-quality, feasible routes from being prematurely pruned by absurd permutations.

6.1.2 Comparative Yield and the Global Optimum

The outputs of these five executions were individually deduplicated and compiled into columnar Parquet databases for quantitative comparison. The metric for success was the discovery of "Elite Routes," defined as any sequence successfully visiting 14 or more sovereign countries within the 24-hour window.

Table 6.1: Comparative Analysis of Search Executions

Execution Strategy	**Search Space Explored**	**Total Elite Routes ($\geq$ 14)**	**16-Country Routes**	**15-Country Routes**	**14-Country Routes**	**Routes with** $\mathrm{RQI} > 0$
Baseline DFS	$\sim$ 1% (Timeout)	114,158	53	3,625	110,480	0
Reduced (`freq100`)	100% (Reduced)	186,933	2,377	30,724	153,832	3
Reduced (`freq1000`)	100% (Reduced)	116,457	1,586	21,645	93,226	3
Distributed (Standard)	100% (Full)	2,901,144	3,658	128,361	2,769,125	0
Distributed (Heuristic)	100% (Full)	1,914,802	2,652	90,135	1,822,015	4

6.1.3 The Theoretical vs. Feasible Ceiling

As demonstrated in Table 6.1, the raw topology of the European network mathematically permits sequences visiting up to 16 countries. However, algorithmically valid paths do not inherently guarantee physical execution. When the Route Quality Index (RQI) was applied to assess real-world human viability, evaluating transfer margins, minimum transit times, and modality constraints, every 15- and 16-country route yielded heavily negative scores. These paths relied on overlapping schedules and physically impossible transfers, proving them to be topological artifacts rather than executable itineraries.
When strictly filtering the final 1.9 million routes for positive viability ($\mathrm{RQI} > 0$), the absolute feasible mathematical ceiling of the European transit network contracted definitively to 14 countries. Extracting these realistic paths required the full computational weight of the heuristic distributed array, which successfully isolated exactly four routing sequences capable of achieving this feasible global optimum.

6.2 Analysis of Results: The Gap Between Topology and Feasibility

The empirical data presented in Table 6.1 captures the fundamental tension at the core of this research: the vast discrepancy between what is mathematically permissible in a graph and what is physically executable in reality. The fluctuations in route volume, maximum country counts, and Route Quality Index (RQI) scores across the five execution phases reveal critical insights into the behavior of the European transit network under extreme combinatorial constraints.

6.2.1 Network Density and the Pareto Principle (`freq100` vs. `freq1000`)

A direct comparison between the two reduced subgraph executions isolates the impact of network density on algorithmic yield. The `freq100` graph, being a larger superset containing thousands of additional secondary transit stations, mathematically generated a substantially higher volume of elite routes than the highly constrained `freq1000` graph (186,933 routes versus 116,457).
However, the application of the RQI reveals a strict Pareto distribution regarding physical feasibility. Despite the `freq100` execution evaluating exponentially more combinations, it yielded the exact same number of physically viable routes (3 routes with $\mathrm{RQI} > 0$) as the smaller `freq1000` graph. This explicitly demonstrates that the feasible, record-breaking capabilities of the European network are entirely reliant on a small fraction of elite arterial hubs. Injecting secondary regional stations into the search space

drastically inflates computational time complexity and topological bloat, but contributes absolutely zero value to the physical optimization of the route.

6.2.2 The B&B "Bullying" Phenomenon (Standard Distributed Execution)

The most striking result in Table 6.1 is the output of the Standard Distributed search. Operating without constraints, it fully parsed the continent, yielding an overwhelming 2,901,144 elite routes. Yet, the physical viability of this dataset was zero.

This failure exposes a critical flaw in applying standard Branch and Bound (B&B) logic to real-world logistics. Because the standard search evaluated chronological connections indiscriminately, it readily utilized physically impossible transfers, such as 1-minute layovers between separate transport networks. These impossible connections allowed the algorithm to artificially pack a massive number of transit legs into the 24-hour window, quickly discovering mathematically valid 15- and 16-country paths early in its execution.

Once these anomalies were discovered, the B&B `global_max` variable was immediately raised to 15 or 16. From that moment forward, the algorithm's bounding logic became ruthlessly efficient but practically destructive. Whenever the search evaluated a highly realistic, human-viable path (which typically necessitates 90-minute safety buffers), the B&B upper-bound calculation correctly determined that the path's maximum potential was capped at 14 countries. Because 14 is strictly less than the inflated `global_max` of 15 or 16, the algorithm pruned the branch. High-quality, perfect-RQI 14-country routes were systematically "bullied" out of the search space and deleted precisely because they adhered to the slower pace of physical reality.

Furthermore, the bloated yield of 2.9 million routes was a direct consequence of these impossible transfers. A path built on 1-minute layovers can squeeze upwards of 30 distinct transport legs into a 24-hour window. This immense depth dramatically increases the graph's branching factor, resulting in an explosion of combinatorial permutations that represent mathematical noise rather than genuine travel alternatives.

6.2.3 Feasibility Extraction via the Heuristic Priority

The transition to the Distributed Heuristic execution resolves the B&B bullying phenomenon and explains the final, definitive numbers of the study.

By restructuring the search to prioritize layovers closest to a 90-minute ideal, the algorithm fundamentally altered the timeline of discovery. It successfully mapped and logged the highly realistic 14-country routes before it eventually uncovered the 15- and 16-country mathematical anomalies. Because the realistic routes were secured prior to the `global_max` being artificially inflated, they survived the execution. This temporal manipulation directly accounts for the successful extraction of the four optimal, human-viable routing sequences ($\mathrm{RQI} > 0$) out of the entire continental graph.

Equally significant is the reduction in total volume, dropping from 2.9 million to 1,914,802 elite routes. Unlike the standard search, the heuristic prioritization forced the algorithm to actively burn the 24-hour clock on realistic waiting periods. A path incorporating multiple 90-minute layovers consumes the available 24 hours in far fewer total transit legs. This reduced depth naturally bounded the combinatorial branching factor, preemptively suppressing the explosion of useless permutations and resulting in a cleaner, vastly more concentrated final dataset.

Ultimately, this analysis proves that while the mathematical topology of the European transit network possesses the density to yield 16-country paths, the strict application of physical feasibility constraints definitively caps the absolute human optimum at 14 countries.

6.3 Methodological Verdict: AoE vs. AoN Paradigms

A primary objective of this research was to conduct a comparative evaluation of graph modeling paradigms for solving highly constrained, multi-objective public transit routing problems. The transition from the

Time-Dependent Activity-on-Edge (AoE) architecture (Chapter 4) to the Activity-on-Node (AoN) Directed Acyclic Graph (Chapter 5) was not arbitrary, but rather a necessary evolutionary step driven by distinct computational bottlenecks. The final results definitively crown the AoN paradigm as the superior architecture for large-scale, exhaustive schedule evaluation.

6.3.1 The Failure of the Time-Dependent AoE Model

The AoE approach was initially selected for its exceptional memory efficiency. By representing transit stations as static vertices and compressing dynamic schedules into temporal arrays on the edges, the model successfully mapped the entire European transit network without triggering Out-Of-Memory (OOM) errors.

However, this spatial compression shifted the entire burden of chronological calculation directly onto the search algorithm. Executing Dynamic Programming (DP) via a Label-Setting algorithm required maintaining Pareto dominance frontiers at every spatial node. The Guinness World Record constraint, minimizing time while maximizing distinct sovereign visits, created a scenario where vast numbers of paths were mathematically incomparable. A path that collected 8 countries by 14:00 could not mathematically dominate a path that collected 9 countries by 16:00.

Consequently, the non-dominated label sets at major transit hubs grew exponentially. The algorithm hit a catastrophic computational "time wall," spending the majority of its CPU cycles executing comparative loops just to establish dominance for newly generated labels.

Crucially, this architecture inherently resisted distributed scaling. Because Label-Setting algorithms rely on constantly evaluating and updating shared global state (the dynamic Pareto frontiers), attempting to deploy this approach across a High-Performance Computing (HPC) cluster would have triggered massive Inter-Process Communication (IPC) latency. The synchronization overhead required to lock and update frontiers across parallel workers would have neutralized any multi-core processing gains. Thus, the AoE approach was theoretically memory-efficient, but practically execution-locked to a single thread, fundamentally failing to map the search space.

6.3.2 The Triumph of the AoN Directed Acyclic Graph

The transition to the AoN architecture required abandoning spatial compression. By transforming specific transit events (e.g., Train 123 from Paris to Brussels at 08:00) into the static nodes themselves, the graph exploded in memory footprint but achieved a vital structural property: it became strictly chronological, forming a Directed Acyclic Graph (DAG).

This paradigm shift solved the exact bottlenecks that killed the AoE approach:

- **Elimination of the Pareto Loop:** In the AoN DAG, chronological time flows in one direction and paths never cyclically cross the exact same transit event. This eliminated the need to maintain expanding Pareto frontiers. Instead of comparing a new label against thousands of existing paths, the DFS algorithm only needed to perform a lightning-fast State Dominance check ($O(1)$ dictionary lookup) against a single benchmark arrival time for that specific node and country set.
- **Decoupled Parallelization:** Because the AoN graph natively enforces chronological flow, it removed the algorithm's reliance on a dynamically shifting global state. The search space could be cleanly severed into discrete, independent starting nodes. This architectural decoupling is what allowed the problem to be successfully deployed across a 100-node SLURM cluster. Each worker process navigated its assigned sector of the DAG completely independently, utilizing local memory dicts to execute state dominance pruning without ever triggering IPC latency.

Ultimately, the comparative study concludes that for multi-objective, time-constrained transit optimization over massive datasets, the memory-heavy Activity-on-Node structure is strictly necessary. While the AoE architecture choked on its own comparative loops, the temporal unwinding of the AoN graph enabled the DFS Branch and Bound parallelization that successfully parsed millions of paths and extracted the feasible global optimum.

6.4 Conclusions and Future Work

6.4.1 Conclusions: The Feasible Optimum vs. Theoretical Topology

The primary objective of this thesis, to construct a comprehensive mathematical model of the European transit network and computationally extract the optimal routing sequence for the Guinness World Record, has been successfully accomplished. By transforming disparate transit schedules into a strictly chronological Activity-on-Node (AoN) Directed Acyclic Graph, and deploying a heuristic-driven distributed Branch and Bound (B&B) search array, the theoretical bounds of the continent's infrastructure were definitively mapped.

The most profound conclusion of this research lies in the staggering disparity between theoretical graph topology and human physical feasibility. The algorithm successfully generated nearly two million unique, mathematically valid itineraries capable of visiting 14, 15, or even 16 sovereign countries within 24 hours. However, the application of the Route Quality Index (RQI) revealed that the vast majority of these topological pathways relied on overlapping schedules and impossible physical transfer margins.

Out of 1.9 million mathematically elite paths, only exactly four distinct routing sequences survived with a positive, human-viable safety margin. This extreme rarity underscores the immense difficulty of the underlying problem. It proves that raw graph routing, when divorced from the harsh realities of physical human logistics, yields data that is mathematically accurate but practically useless. By systematically filtering out these mathematical anomalies, this research conclusively establishes that the absolute, physically feasible human ceiling for this endeavor is 14 countries.

6.4.2 Algorithmic Enhancements: The Layered Capping Search

While the heuristic distributed search successfully extracted the feasible global optimum, the interaction between the B&B pruning logic and the RQI presents a clear avenue for algorithmic optimization. As analyzed in Section 6.2, the aggressive nature of the B&B bounding function frequently pruned highly realistic, perfect-RQI 14-country routes because they were mathematically incapable of matching the physically impossible 15-country mathematical anomalies discovered earlier in the execution.

A primary direction for future algorithmic work is the implementation of a Layered Capping Search. By intentionally capping the `global_max` bounding variable at the known human limit (14 countries), the algorithm would be forced to disable its aggressive upper-bound pruning for any branch capable of reaching that tier. This would effectively convert the algorithm from a pure maximum-hunting search into an exhaustive enumerator for the 14-country space. While this would drastically increase the computational time required to evaluate that specific layer, it would guarantee the discovery and preservation of every single high-RQI, human-viable route that was previously "bullied" out of the search space by impossible 15-country trajectories.

6.4.3 Multi-Objective Attributes and Stochastic Modeling

Beyond graph traversal mechanics, the routing engine possesses the foundational architecture to support complex multi-objective optimization. The current Route Quality Index evaluates feasibility purely as a function of temporal transit margins. Future iterations of this engine could integrate additional dimensions, optimizing routes for minimal carbon footprint (CO_2 emissions per passenger kilometer), dynamic ticket pricing, or stochastic delay modeling.

Moving from a deterministic graph, where transit operates strictly on scheduled times, to a probabilistic graph would allow the algorithm to calculate the statistical likelihood of a route's success. However, while theoretically sound, integrating these dimensions requires access to highly restricted, fragmented, and often proprietary commercial datasets (such as historical airline delay APIs and dynamic rail pricing endpoints). Consequently, these enhancements were deliberately deferred to future work due to the inherent time constraints and data-access limitations of this academic scope, rather than algorithmic inability.

6.4.4 Broader Applications: A Pan-European Multi-Modal Engine

While the specific algorithm developed for this thesis targets a highly niche Guinness World Record application, the underlying architecture holds immense broader value. The most significant achievement of this research is the data engineering pipeline itself: the successful parsing, UTC-synchronization, and topological normalization of disparate terrestrial feeds and aviation data into a singular, cohesive Directed Acyclic Graph.

This unified data pipeline forms the core technological foundation required to build a comprehensive, consumer-facing multi-modal routing engine, effectively a "Google Maps" for integrated European transit. While commercial flight aggregators and standalone rail planners are ubiquitous, cross-modal engines capable of seamlessly bridging aviation and complex regional rail networks remain a profound industry challenge. The architecture developed herein proves that such an integration is topologically possible and computationally navigable, opening the door for future deployment via a Graphical User Interface (GUI) or commercial API.

6.4.5 Final Remarks

Through rigorous data normalization, architectural pivoting, and distributed parallelization, the mathematical dimensions of the European transit network have been solved. The theoretical exploration of this routing problem is now concluded. The final remaining step in this endeavor is to step away from the computational model and attempt the physical execution of the optimal route. The logistical and adjudication frameworks for this physical Guinness World Record attempt are detailed in Appendix A.

A
Supporting Material

[Additional tables and data will go here]